Especificação de Produto & Arquitetura — Portal PIBLS

GUIA DE REFERÊNCIA PARA CRIAÇÃO DO PRD.MD E ARCHITECTURE.MD • MIGRAÇÃO WORDPRESS PARA MONOLITO

PRD.md + ARCHITECTURE.md
HostGator • React + PHP 8 + MySQL

PARTE 1 • PRD.md

VISÃO DO PRODUTO, ESCOPO PÚBLICO E HUB DIGITAL DA IGREJA

Product Requirements Document

1. Objetivo Executivo do Produto: Substituir o antigo site em WordPress da **Primeira Igreja Batista em Lagoa Santa (pibls.com)** por uma aplicação web monolítica moderna, rápida, dinâmica e profissional, com **orçamento adicional zero** (mantendo a hospedagem compartilhada e o e-mail profissional na HostGator). O novo portal funcionará simultaneamente como o **Portfólio Institucional Completo** para visitantes e como a **Rede Central de Guias, Eventos e Ferramentas Tecnológicas** para todos os membros, congregados e os 20 ministérios da igreja.

PÚBLICO 01: VISITANTES
Pessoas buscando conhecer como a igreja funciona, história desde 1985, horários de cultos, localização em Lagoa Santa ("Como Chegar"), recepção e contato rápido via WhatsApp ou formulário.

PÚBLICO 02: MEMBROS & MINISTÉRIOS
Rede ativa para leitura e download de boletins, inscrição em eventos especiais, acesso às Células de CL por bairros, agenda da Juventude (Salt), MCM, UMHBB, Mini PIB e ofertas via PIX.

PÚBLICO 03: ADMINISTRAÇÃO
Liderança, secretaria e comunicação operando um Painel Administrativo próprio, com usuários e permissões provisionados diretamente no banco MySQL via cPanel da HostGator.

2. MATRIZ DE REQUISITOS FUNCIONAIS — PORTAL PÚBLICO (FRONTEND INSTITUCIONAL)

ID	MÓDULO / PÁGINA	DESCRIÇÃO DETALHADA DOS REQUISITOS (BASEADO NO GUIA INSTITUCIONAL PIBLS)
RF-01	**Início & Programação**	Vitrine dinâmica de boas-vindas, destaque para a campanha *"Estamos Construindo"*, aviso da Ceia do Senhor (1º domingo do mês) e quadro de horários: Quinta (20h), Domingo EBD (9h), Culto Matinal (10h10), Culto Noturno (19h30) e Culto Salt (1ª sexta às 19h30).
RF-02	**Conheça a Igreja (História)**	Linha do tempo interativa dos 40 anos (fundação em 13/01/1985 no bairro Promissão, apoio norte-americano, desabamento em 2018 e reconstrução), galeria de ex-pastores, afiliação à CBB (Declaração de Fé) e biografia completa do **Pr. Joselito F. Sena**.
RF-03	**Guia dos 20 Ministérios**	Catálogo vivo de como a igreja funciona, listando atribuições e líderes atuais: Comunicação, Oração, Tesouraria, Diáconos, Patrimônio, Recepção, Juventude, Música/Sonoplastia, Multimídia/Transmissão, Missões, Cantina, Ação Social, EBD, Infantil, MCM, UMHBB, Embaixadores e Mensageiras do Rei.
RF-04	**Rede "Para Você" & Grupos**	Diretório de integração por perfil (1ª Pedro 4:10): mapa/lista de **Células de CL** nos lares (terças e quartas às 20h), Homens (último sábado às 20h), Mulheres (3º sábado às 17h30), Jovens/Adolescentes (Salt) e Crianças (Mini PIB: domingo 9h e 19h30).
RF-05	**Central de Boletins**	Repositório público de todos os boletins informativos semanais/mensais cadastrados no painel, com busca por data, leitura responsiva na tela e opção de download em PDF.
RF-06	**Área Especial PIX & Ofertas**	Componente interativo para dízimos, ofertas e construção (Favorecido: PIBLS): exibição do QR Code oficial, botão *"Copiar Chave PIX"* com 1 clique (**CNPJ: 20969119000184**) com feedback visual e guia passo a passo para pagamento nos aplicativos bancários.
RF-07	**Eventos & Ferramentas**	Motor de páginas para eventos especiais, congressos, acampamentos e ações ministeriais que exijam recursos tecnológicos: formulários dinâmicos de

PAINEL ADMINISTRATIVO, LEADS E PERMISSÕES VIA CPANEL

5. Especificação do Painel Administrativo (Backoffice do Monolito): O sistema contará com uma área administrativa restrita (`/admin` no frontend React comunicando-se com a API PHP) para operação diária da igreja. Por requisito estrito de governança e segurança, **nenhum usuário ou nível de permissão é criado por telas públicas**: toda a criação de contas e atribuição de permissões ocorre diretamente no banco de dados MySQL gerenciado pelo **cPanel da HostGator**, cabendo à API realizar a leitura (`GET/SELECT`) desses registros para autenticar e autorizar ações no painel.

6. MATRIZ DE REQUISITOS FUNCIONAIS — PAINEL ADMINISTRATIVO E AUTENTICAÇÃO

ID	MÓDULO ADMINISTRATIVO	REGRA DE FUNCIONAMENTO E ESCOPO OPERACIONAL
`ADM-01`	**Leitura de Usuários e Permissões (cPanel → API)**	O administrador técnico cadastra os usuários e suas permissões diretamente nas tabelas do MySQL via cPanel (phpMyAdmin). A API PHP executa `GET /api/admin/users` e valida a sessão lendo os papéis já provisionados no banco.
`ADM-02`	**Diretório de Usuários no Painel**	Tela no Painel Administrativo que lista todos os usuários criados no banco, exibindo nome, e-mail, ministério vinculado, status (ativo/inativo) e matriz de permissões carregada via `GET` da API.
`ADM-03`	**Caixa de Entrada de Leads e Mensagens**	Centralização de todas as mensagens enviadas através dos formulários do site (pedidos de oração, *"Quero ser membro"*, dúvidas de visitantes, contato com ministérios). Além de disparar notificação para o e-mail institucional, grava cada lead no MySQL com status (Novo, Em Atendimento, Concluído).
`ADM-04`	**Gestão de Eventos e Inscrições**	CRUD completo para criar eventos especiais (título, data, local, banner, vagas, campos do formulário de inscrição) e painel para visualizar/exportar (CSV/PDF) a lista de inscritos em cada evento.
`ADM-05`	**Publicador de Boletins e Devocionais**	Módulo para cadastrar os boletins semanais (edição, data, resumo em texto e upload do PDF para pasta pública controlada) e publicar mensagens pastorais (quadro *Palavra Viva*).
`ADM-06`	**Atualização de Ministérios e Horários**	Interface para atualizar nomes de coordenadores dos 20 ministérios, endereços/dias das Células de CL, horários de cultos e dados da chave PIX/QR Code sem precisar alterar código-fonte.

FLUXO DE GOVERNANÇA DE ACESSO: DO CPANEL (HOSTGATOR) PARA A APLICAÇÃO

- **Passo 1 (Provisionamento no cPanel):** O responsável acessa o *cPanel da HostGator → phpMyAdmin* e insere/atualiza o registro na tabela `users` e os vínculos na tabela `user_permissions` (ex.: `role = 'COMUNICACAO'`, permissões `['boletins.write', 'eventos.write', 'leads.read']`).
- **Passo 2 (Autenticação Segura):** O usuário faz login no painel. O backend PHP verifica o hash de senha (`password_verify`) no MySQL via PDO e inicia sessão com cookies `HttpOnly`, `Secure` e `SameSite=Strict`.
- **Passo 3 (Consumo via GET):** O frontend React consulta `GET /api/auth/me` e `GET /api/admin/users` para renderizar dinamicamente apenas os módulos permitidos ao perfil daquele líder ou secretário.

7. TIPOS DE FORMULÁRIOS CAPTURADOS (LEADS)

- **Ficha de Visitante / Boas-Vindas:** Nome, WhatsApp, bairro e se deseja receber visita ou conhecer uma Célula de CL.
- **"Quero Ser Membro" / Batismo:** Pré-cadastro direcionado ao pastor e à secretaria para classe de novos membros.
- **Pedidos de Oração:** Encaminhados automaticamente para a liderança do Ministério de Oração.
- **Inscrição em Eventos:** Formulários específicos para [illegible]

8. REQUISITOS NÃO-FUNCIONAIS (RNF)

- **RNF-01 (Custo Zero de Infraestrutura):** Execução 100% compatível com o plano compartilhado atual da HostGator (Apache + PHP 8.2 a 8.5 + MySQL).
- **RNF-02 (Preservação de E-mail):** Zero alteração de nameservers ou registros MX/SPF/DKIM durante a substituição do WordPress.
- **RNF-03 (Segurança de Dados):** Credenciais fora do `public_html`, consultas parametrizadas com PDO e proteção CSRF nos formulários.

PARTE 2 • ARCHITECTURE.md

ARQUITETURA MONOLÍTICA: REACT + PHP 8 + MYSQL NA HOSTGATOR

System Architecture Document

1. Visão Arquitetural: Aplicação estruturada como um **Monolito Modular** hospedado integralmente na conta atual da HostGator. O frontend em **React + Vite + TypeScript** é compilado localmente em arquivos estáticos e servido na raiz pública (`public_html/`), enquanto o backend em **PHP 8.x (API REST)** responde sob o subdiretório `public_html/api/`, conectando-se via **PDO** ao banco **MySQL** provisionado no cPanel e lendo variáveis sensíveis em diretório privado fora da raiz web.

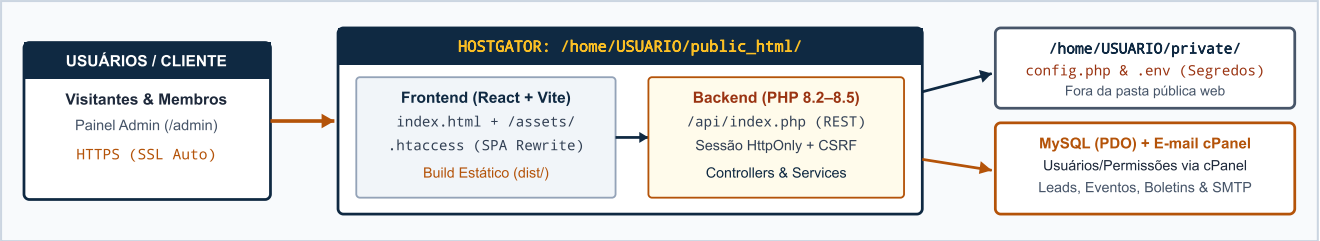


Figura 1: Topologia do monolito na hospedagem compartilhada HostGator com isolamento de credenciais fora do public_html.

CAMADA FRONTEND

React + Vite + TypeScript: Compilado via `npm run build` localmente. Apenas o conteúdo de `dist/` vai para o servidor (jamais enviar `node_modules`).

CAMADA BACKEND (API)

PHP 8.2 a 8.5 + REST: Rotas centralizadas, autenticação por sessão com cookies seguros, validação de entrada, disparo de e-mail e lógica de negócio.

CAMADA DE PERSISTÊNCIA

MySQL + PDO: Consultas 100% parametrizadas (Prepared Statements), migrações SQL versionadas e gestão de usuários/permissões via cPanel.

2. ESTRUTURA DO REPOSITÓRIO (DESENVOLVIMENTO LOCAL)

```
pibls-portal/
├── PRD.md                    # Requisitos de Produto
├── ARCHITECTURE.md           # Guia de Arquitetura e Deploy
├── frontend/                 # Aplicação React + Vite + TS
│   ├── src/                  # Páginas, Componentes, Admin
│   ├── public/               # Imagens, QR Code PIX, Ícones
│   ├── package.json
│   └── vite.config.ts
├── backend/                  # API REST em PHP 8.x
│   ├── public/index.php      # Ponto de entrada público da API
│   ├── src/
│   │   ├── Controllers/      # Auth, Users, Leads, Eventos, Boletins
│   │   ├── Services/         # Regras de negócio e Mailer
│   │   ├── Repositories/     # Queries PDO parametrizadas
│   │   └── Middleware/       # AuthSession, RoleCheck, Csrf
│   ├── config/               # Mapeamento de variáveis
│   ├── routes/api.php        # Definição de endpoints REST
│   ├── database/migrations/  # Scripts SQL (schema inicial)
│   └── composer.json
└── .gitignore                # Ignora node_modules, .env, dist
```

3. ESTRUTURA DE PUBLICAÇÃO NO SERVIDOR HOSTGATOR

```
/home/SEU_USUARIO/
├── private/                  # FORA DO ACESSO WEB PÚBLICO
│   ├── config.php            # Credenciais MySQL e SMTP
│   ├── .env                  # Variáveis de ambiente de produção
│   └── backend_core/         # src/, vendor/, routes/ (protegidos)
│
└── public_html/              # RAIZ PÚBLICA DO DOMÍNIO (pibls.com)
    ├── index.html            # Build do React (gerado em dist/)
    ├── assets/               # JS/CSS minificados pelo Vite
    ├── uploads/
    │   └── boletins/         # PDFs de boletins e banners
    ├── .htaccess             # Roteamento SPA React + HTTPS
    └── api/
        ├── index.php         # Front Controller (chama /private/)
        └── .htaccess         # Rewrite de rotas para api/index.php
```

ISOLAMENTO DA RAIZ PÚBLICA E SEGURANÇA NO APACHE (.HTACCESS)

BANCO DE DADOS, ROTAS DA API, DEPLOY E PRESERVAÇÃO DE E-MAIL

4. MODELAGEM DO BANCO MYSQL (CPANEL) E CONTRATO DE ENDPOINTS DA API REST

TABELA MYSQL	COLUNAS PRINCIPAIS E ORIGEM DOS DADOS	ENDPOINTS REST CORRESPONDENTES (PHP 8 + PDO)
`users`	`id, name, email, password_hash, ministry, is_active` (Criados diretamente via cPanel/MySQL).	`POST /api/auth/login` • `POST /api/auth/logout` `GET /api/admin/users` (Lista todos os usuários do banco)
`user_permissions`	`id, user_id, role_code, permission_key` (Permissões atribuídas via cPanel/MySQL).	`GET /api/auth/me` (Retorna usuário logado + array de permissões carregadas do banco para o painel)
`leads_messages`	`id, form_type, sender_name, phone, email, message, status, created_at` (Formulários do site).	`POST /api/public/leads` (Salva no banco + envia e-mail) `GET /api/admin/leads` • `PATCH /api/admin/leads/{id}`
`events`	`id, slug, title, description, event_date, location, max_spots, form_schema_json, is_published`.	`GET /api/public/events` • `POST /api/admin/events` `PUT /api/admin/events/{id}` • `DELETE /api/admin/events/{id}`
`event_registrations`	`id, event_id, participant_name, phone, email, custom_answers_json, created_at`.	`POST /api/public/events/{slug}/register` `GET /api/admin/events/{id}/registrations`
`bulletins`	`id, edition_number, reference_date, title, summary_text, pdf_url, published_by`.	`GET /api/public/bulletins` `POST /api/admin/bulletins` (Upload seguro de PDF)
`ministries_groups`	`id, category, name, leaders_text, schedule_text, neighborhood, contact_phone` (20 ministérios e CLs).	`GET /api/public/ministries` `PUT /api/admin/ministries/{id}`
`site_settings`	`setting_key, setting_value` (Chave PIX CNPJ `20969119000184`, payload QR Code, links YouTube).	`GET /api/public/settings` `PUT /api/admin/settings`

5. PASSO A PASSO DE PUBLICAÇÃO (DEPLOY NA HOSTGATOR)

1. **Inventário no cPanel:** Acessar `pibls.com/cpanel`, confirmar PHP (8.2 a 8.5), extensões `pdo_mysql`, espaço em disco, SSL ativo e caminho físico de `/home/USUARIO/`.
2. **Banco MySQL e Usuários:** Pelo Assistente de Bancos de Dados do cPanel, criar o banco e o usuário da aplicação. Executar as migrações SQL no phpMyAdmin e popular as tabelas `users` e `user_permissions`.
3. **Configuração Privada:** Criar a pasta `/home/USUARIO/private/` e salvar `config.php` / `.env` com as credenciais do banco e SMTP.
4. **Build do Frontend:** Executar `npm run build` no projeto React + Vite local e enviar o conteúdo da pasta `dist/`

6. PADRÕES DE SEGURANÇA OBRIGATÓRIOS

- **Sessões e Cookies:** Autenticação no PHP configurada com `session_set_cookie_params(['httponly' => true, 'secure' => true, 'samesite' => 'Strict'])`.
- **Proteção contra CSRF e SQL Injection:** Token CSRF validado em todas as rotas `POST/PUT/PATCH/DELETE` e uso exclusivo de `PDO::prepare()` com parâmetros tipados.
- **Upload Restrito de Boletins:** Validação de MIME type real (`application/pdf`, imagens), limite de tamanho e renomeação segura de arquivos antes de mover para `public_html/uploads/boletins/`.
- **Bloqueio de Listagem:** `Options -Indexes` no `.htaccess` para impedir navegação direta em pastas.